INF6083 – Systèmes de recommandation

Projet P3 : Filtrages basés sur : la collaboration et la connaissance

Hiver 2026

Enseignant	Courriel	Local
Etienne Tajeuna	etiennegael.tajeuna@uqo.ca	A2222

Département d'Informatique et d'Ingénierie (DII)

31 mars 2026

Sommaire

Ce troisième projet fait suite au projet P2 (recommandation basée sur le contenu) et au projet P1 (analyse structurelle des graphes d'interaction). L'objectif est de compléter la trilogie des paradigmes fondamentaux en introduisant le **filtrage collaboratif** et le **filtrage basé sur la connaissance** puis de comparer les résultats.

Table des matières

1 Tâches du projet **1**

1.1 Tâche 0 : Filtrage basé sur le contenu 1

1.2 Tâche 1 : Filtrage collaboratif 1

1.3 Tâche 2 : Système de recommandation basé sur les graphes de connaissances 1

2 Livrables **4**

2.1 Format de remise 4

1 Tâches du projet

1.1 Tâche 0 : Filtrage basé sur le contenu

À partir d'un sous-ensemble de données D de votre choix (un ensemble facilement acceptable par les ressources de votre votre machine) évaluer votre système de recommandation basé sur le contenu obtenu dans le TP2.

1.2 Tâche 1 : Filtrage collaboratif

En exploitant le sous-ensemble de données D, concevoir un système de recommandation basé sur un type de collaboration de votre choix (collaboration item ou collaboration utilisateur) et l'évaluer.

1.3 Tâche 2 : Système de recommandation basé sur les graphes de connaissances

Dans cette tâche, vous êtes amenés à concevoir un système de recommandation exploitant un graphe de connaissances, en intégrant des mécanismes d'inférence et d'interrogation.

1. **Mise en place de l'environnement**
   Installer les outils nécessaires à la manipulation de graphes RDF et à leur interrogation. Vous pouvez, par exemple, utiliser *Protégé* (`https://protege.stanford.edu`) pour la modélisation de l'ontologie et un triplestore tel que *Apache Jena Fuseki*(`https://jena.apache.org/documentation/fuseki2/`) ou *GraphDB* (`https://graphdb.ontotext.com/`) ou tout autre outils libre de votre choix pour le stockage et l'interrogation des données.
   Vous devrez valider votre installation en :
   — créant un premier graphe RDF simple ;
   — exécutant une requête SPARQL basique.
2. **Conception du graphe de connaissances**
   À partir du jeu de données D, concevoir un graphe de connaissances en identifiant :
   — les entités principales (par exemple : utilisateurs, items, catégories, auteurs, marques ou mots-clés) ;
   — les relations entre ces entités ;
   — les propriétés descriptives associées.
   Vous devrez proposer une modélisation explicite de votre graphe (schéma, ontologie ou diagramme) en justifiant vos choix.
3. **Construction et stockage des triplets**
   Transformer votre modélisation en un ensemble de triplets RDF de la forme (sujet, prédicat, objet).
   Charger ces triplets dans un triplestore et vérifier leur bonne intégration en :
   — explorant le graphe ;

— exécutant des requêtes SPARQL simples permettant de retrouver des informations de base.

4. **Définition de règles d'inférence**
   Proposer un ensemble de règles permettant d'enrichir votre graphe de connaissances. Ces règles peuvent être exprimées :
   — soit via des axiomes OWL ;
   — soit via des règles de type SWRL ;
   — soit via un mécanisme de règles propre au moteur utilisé.
   Vos règles doivent permettre de déduire de nouvelles connaissances utiles à la recommandation.
   Exemple illustratif :
   Supposons les triplets suivants :
   ```
   :User1 :likesCategory :ScienceFiction .
   :BookA :hasCategory :ScienceFiction .
   :BookA :writtenBy :AuthorX .
   :BookB :writtenBy :AuthorX .
   ```

   Une première règle peut consister à inférer qu'un item appartenant à une catégorie appréciée par un utilisateur devient un item potentiellement recommandable :
   ```
   IF (?u :likesCategory ?c) AND (?i :hasCategory ?c)
   THEN (?u :mayLike ?i)
   ```

   Une deuxième règle peut propager un intérêt vers d'autres items reliés au même auteur :
   ```
   IF (?u :mayLike ?i1) AND (?i1 :writtenBy ?a) AND (?i2 :writtenBy ?a)
   THEN (?u :mayLike ?i2)
   ```

   Dans cet exemple, même si l'utilisateur n'a jamais interagi directement avec `:BookB`, celui-ci peut être recommandé par inférence.

5. **Exploitation des inférences et interrogation avancée**
   Activer le mécanisme d'inférence du triplestore et analyser les nouvelles connaissances générées.
   En exploitant SPARQL, formuler des requêtes permettant :
   — d'extraire des informations explicites ;
   — d'extraire des informations inférées ;
   — de produire des recommandations basées sur ces résultats.
   Les requêtes suivantes sont données à titre de gabarit et devront être adaptées à votre modélisation.
   Exemple 1 : retrouver les catégories appréciées par un utilisateur
   ```
   PREFIX : <http://example.org/reco#>

   SELECT ?category
   WHERE {
       :User1 :likesCategory ?category .
   }
   ```

Exemple 2 : retrouver les items explicitement liés à une catégorie appréciée

```
PREFIX : <http://example.org/reco#>

SELECT ?item
WHERE {
    :User1 :likesCategory ?category .
    ?item :hasCategory ?category .
}
```

Exemple 3 : retrouver les items inférés comme potentiellement appréciés

```
PREFIX : <http://example.org/reco#>

SELECT ?item
WHERE {
    :User1 :mayLike ?item .
}
```

Exemple 4 : exclure les items déjà consommés ou déjà évalués

```
PREFIX : <http://example.org/reco#>

SELECT ?item
WHERE {
    :User1 :mayLike ?item .
    FILTER NOT EXISTS { :User1 :hasInteractedWith ?item . }
}
```

Exemple 5 : comparer les résultats obtenus avec et sans inférence

Vous devrez exécuter une même requête ou deux requêtes proches :

- une première sur les relations explicitement présentes dans la base ;
- une seconde après activation de l'inférence, en observant les nouveaux triplets accessibles.

Par exemple :

```
PREFIX : <http://example.org/reco#>

SELECT ?item
WHERE {
    :User1 :mayLike ?item .
}
```

Si la relation `:mayLike` n'a pas été explicitement insérée mais est produite par vos règles, alors les résultats obtenus illustreront directement l'apport de l'inférence.

6. **Évaluation et comparaison**

   Définir une métrique d'évaluation pertinente pour votre système de recommandation.

Évaluer votre approche basée sur le graphe de connaissances et comparer les résultats obtenus avec ceux des approches précédentes :

— filtrage basé sur le contenu (Tâche 0) ;
— filtrage collaboratif (Tâche 1).

Vous devrez analyser les différences observées et discuter :

— de l'apport de la représentation sémantique ;
— de l'impact de l'inférence sur les recommandations obtenues ;
— des avantages et limites de cette approche par rapport aux deux précédentes.

2 Livrables

Le projet doit être réalisé par le même groupe que pour P1 et P2 (3 à 4 étudiants).

Date de remise : 05 avril 2026 au plus tard à 23h59.

2.1 Format de remise

Soumettez un fichier compressé nommé `INF6083-P3-EquipeN.zip` contenant :

1. **Rapport (PDF)** : `INF6083-P3-EquipeN-Rapport.pdf`
   — maximum 5 pages, hors annexes ;
   — structure claire suivant les tâches du projet ;
   — tableaux comparatifs des performances (contenu, collaboratif, connaissance) ;
   — discussion sur la complémentarité des approches.
2. **Code source** : `INF6083-P3-EquipeN-Code.ipynb` ou fichiers `.py`
   — code structuré et commenté ;
   — instructions d'exécution claires ;
   — fichier `requirements.txt` ;
   — fichier `README.md`.